



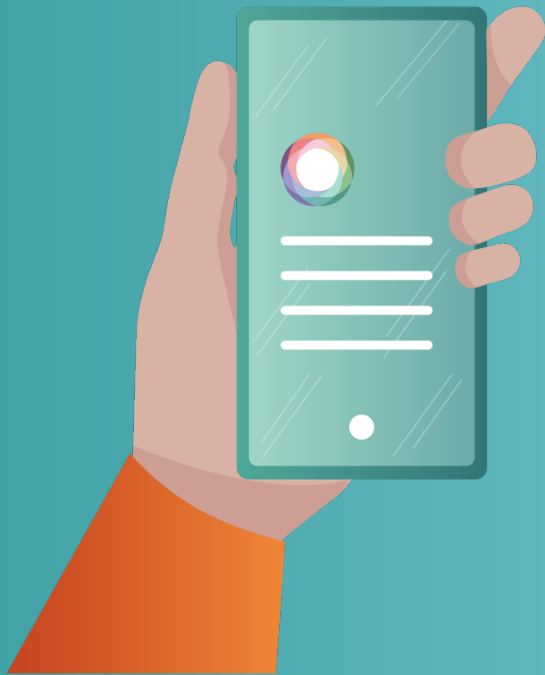
Open Payments

Third-party Account Access

Nathan Lie

Last update: 19 June 2026

Original slide credit: Sarah Jones



Open Payments (OP)

→ Open Payments is a standard for making payments online

 Interledger
test boutique

Products Shopping Cart

Select payment method

 Interledger

 Apple Pay

 G Pay

Wallet Address

Pay with  Interledger

Order summary



Luck

Qty: 1

Price per unit: \$77.00

\$77.00

Subtotal

\$77.00

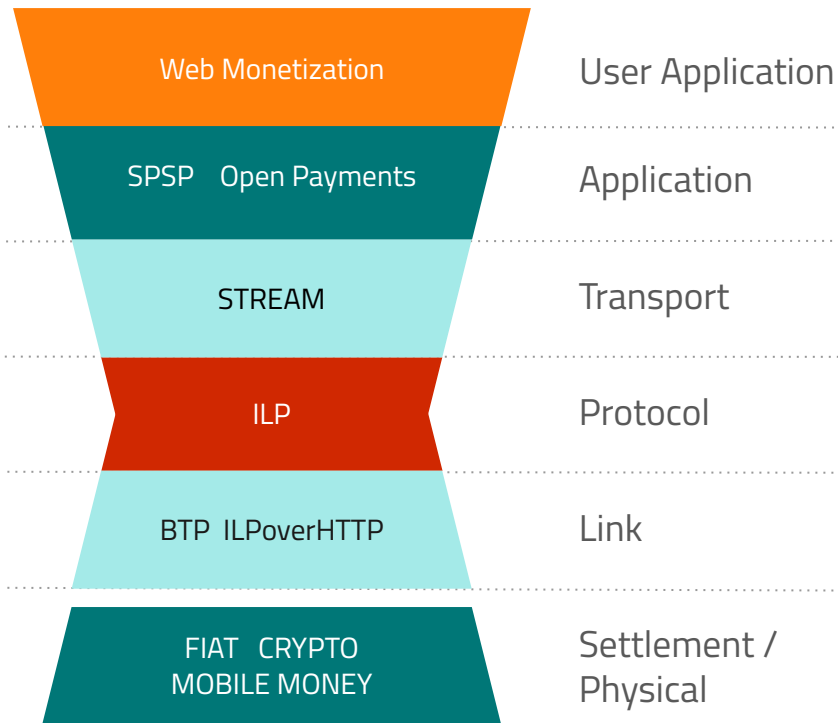
Fees

Fees will be calculated at the next step

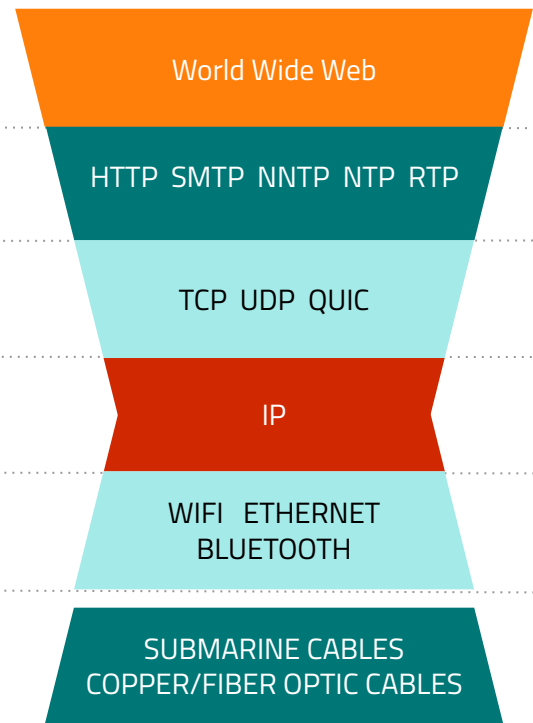
Estimated order total

\$77.00

Interledger

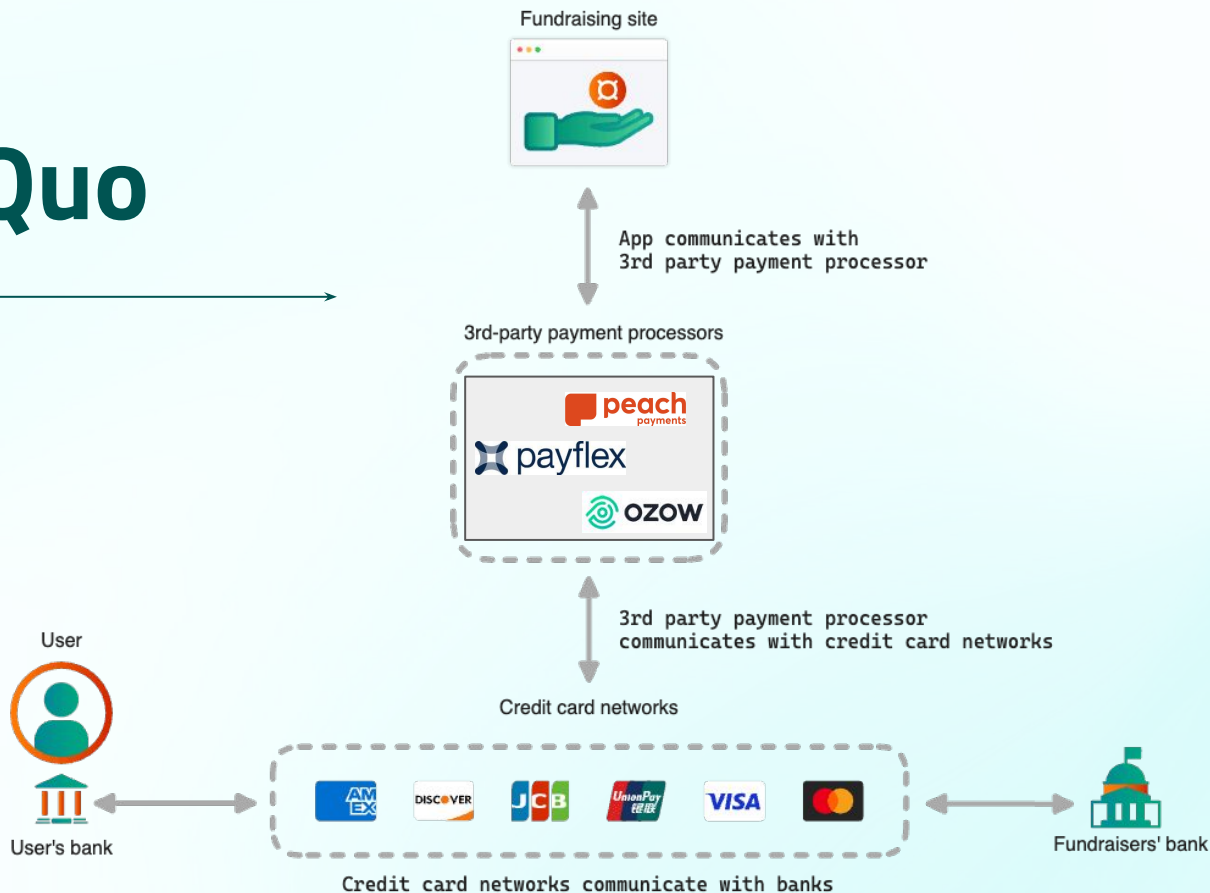


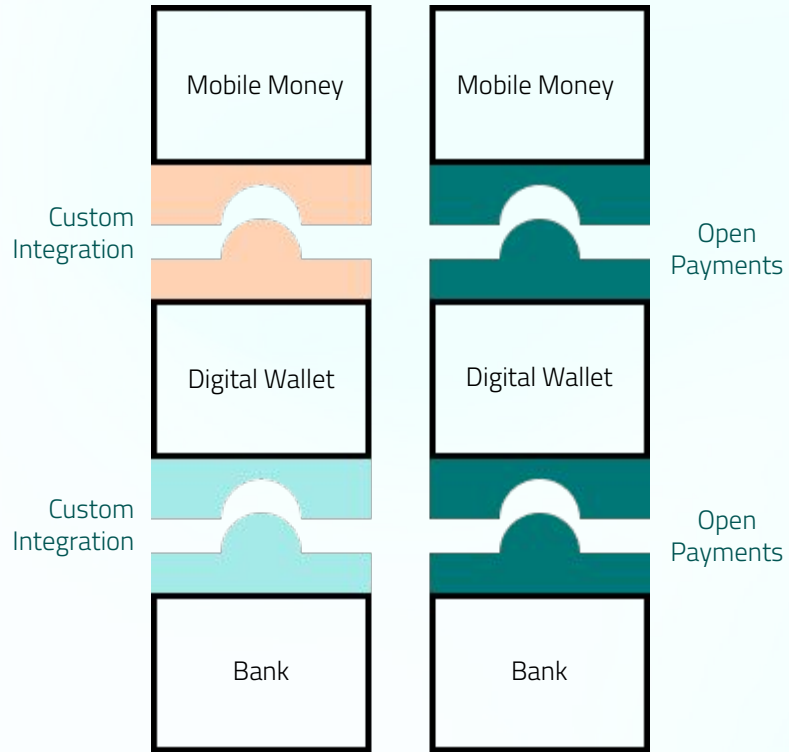
Internet



The Status Quo

- Intermediaries
- Direct integrations
- Complex scenarios





Where Open Payments Comes In

Standardisation

→ One standard to enable **direct access** to any account.

Open Payments (OP)

What It Is

- Open Payments give online applications direct and secure communication with user's accounts
- User's remain in control

Open Payments (OP)

Why It Matters

Send a payment as easily as you send an email

- For any account type,
 - ◆ Bank, mobile money provider, digital wallet provider etc.
- For any currency
- In any location
- No sensitive information is shared or stored



Interlude: Wallet Address

- Every Interledger-enabled account is identified by one or more URLs called wallet addresses
- A wallet address is an alias for the underlying financial account.
- A wallet address is a human readable, publicly shareable account identifier
- It is also a service endpoint that provides the entry point for the API.
- The sender and recipient must both have a wallet address to transact using Open Payments.

What is Open Payments (OP)

How it works

- RESTful API: Application Programming Interface
- Defines how different services communicate with each other
- How It Works:
 - ◆ Request: Online applications ask the account provider for a quote or to make a payment.
 - ◆ Response: The provider replies with the quote or the payment status.

Who uses Open Payments (OP)

Account Servicing Entities (ASEs)

implement the OP standard

- Write the code to implement the functionality behind the API requests.
- ASE becomes OP-enabled
- E.g. When a request is received to make a quote, gather all necessary information, calculate any fees, determine validity period, add a new entry to the database, etc.

Third party applications call the OP APIs

- Use OP to interact with customer accounts.
- Connect to any ASE using OP without custom integrations.
- E.g. send a request to make a quote for a customer who wants to download a song

Open Payments (OP) Benefits

A user's perspective

- Users can:
- ◆ Remain in full control
 - ◆ See fees upfront in their own currency
 - ◆ Enjoy a simple checkout flow
 - ◆ Use recognizable wallet address
 - ◆ Keep sensitive information safe
 - ◆ Preserve their privacy
 - ◆ Grant specific access to their accounts

Open Payments (OP) Benefits

A developer's perspective

- Developers can build payment functionality:
 - ◆ Without third-party payment processors
 - ◆ Without fees
 - ◆ Without custom integrations
 - ◆ Without storing sensitive information
 - ◆ In any programming language
 - ◆ While retaining control over user experience
 - ◆ That supports any kind of account, globally

Open Payments (OP) Benefits

An account provider's perspective

- Account providers can
 - ◆ Align with Open Banking and Open Finance trends
 - ◆ Expand across currencies and countries, using your existing infrastructure
 - ◆ Reduce operational overhead by replacing bespoke integrations with standardized protocols
 - ◆ Open new revenue streams for your users like digital tipping, streaming payments, or pay-per-use models.

Open Payments (OP) Benefits

A regulator's perspective

- An open, non-proprietary API
 - ◆ Introduces pro-competition interoperability
 - ◆ Reduces vendor lock-in
 - ◆ Enables portability
 - ◆ Improves transparency for users about where their money is going
 - ◆ Increases digital financial inclusion
 - ◆ Ensures a level playing field between established institutions and newer startups

Messaging, Clearing & Settlement

Behind the scenes

- **Enables apps to trigger payments directly from user accounts.**
 - ◆ Messaging → Exchanging payment instructions (who pays whom, how much).
 - ◆ Clearing → Confirming obligations between accounts.
 - ◆ Settlement → Moving the actual funds to finalize payment.
- Open Payments includes messaging and clearing but not settlement
- For a payment to succeed, both providers need to be Open Payments-enabled and have a way to settle between them



Bob

I'd like to open an account

Fill in KYC info

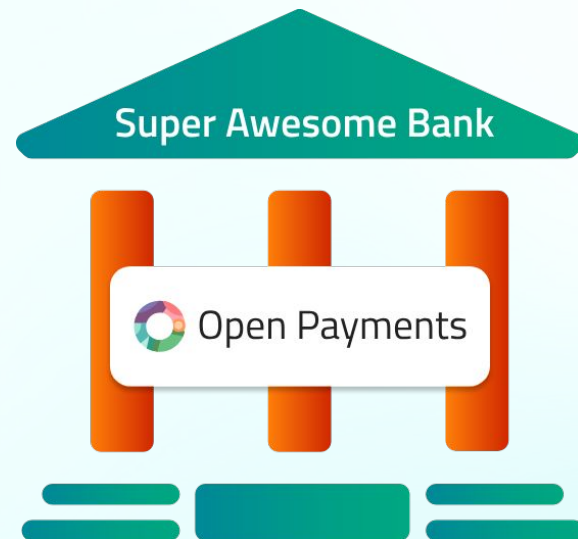




Bob

Acc num: 1765908643

Wallet address:
<https://bank.com/bob>



Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment

Online Marketplace



Customer



ASE



Customer



ASE

- Customer: end-users
- Client: app that processes payments by making OP API requests
- ASE: entity that manages customers' accounts and implements the OP standard.

Deep Dive: Open Payment Flows

Who's Who

→ Client application

→ Customers

→ ASE

- ◆ Wallet address server
- ◆ Resource server (RS)
- ◆ Authorization server (AS)
- ◆ Identity provider (IdP)

Client



Customer / Sender



Customer / Recipient

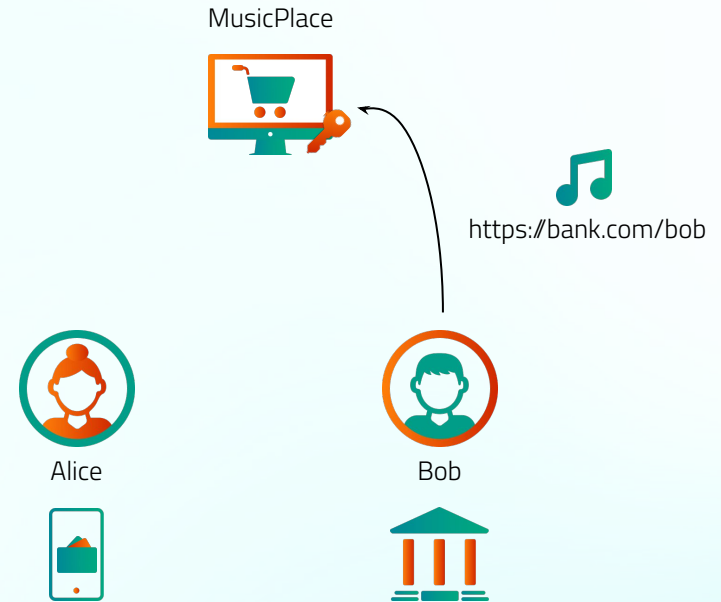


ASE

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment



Three participants:

- Bob adds a song to MusicPlace
- Provides his wallet address to be paid when someone downloads the song

Interlude: Wallet Address

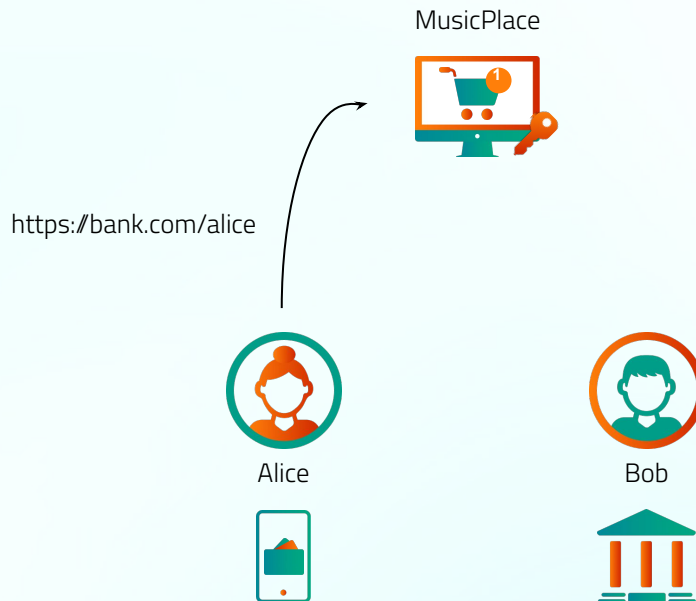
```
curl -H "Accept: application/json" https://ilp.interledger-test.dev/sarah
```

```
{  
  "id": "https://ilp.interledger-test.dev/sarah",  
  "publicName": "SarahEuro",  
  "assetCode": "EUR",  
  "assetScale": 2,  
  "authServer": "https://auth.interledger-test.dev",  
  "resourceServer": "https://ilp.interledger-test.dev"  
}
```

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment

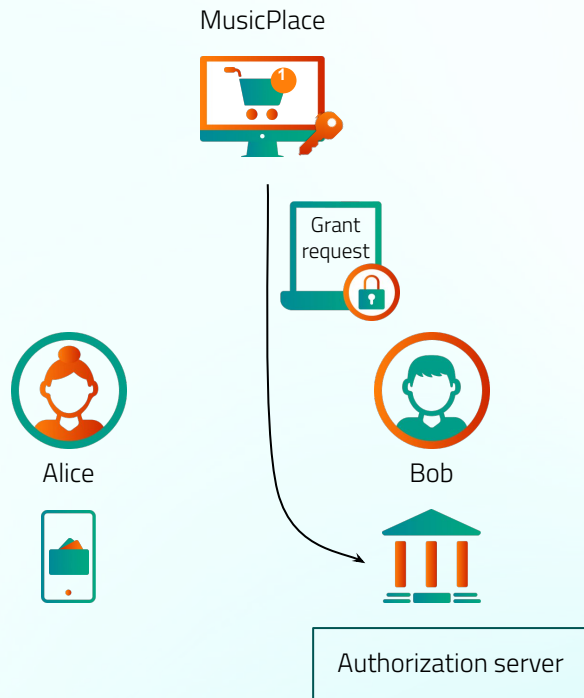


- Alice adds Bob's song to her cart
- Provides her wallet address to pay for the song using Open Payments

Open Payments

Third-Party Access to Accounts

- **Grant Request Incoming Payment**
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment



- MusicPlace requests a **Grant** to create an **Incoming Payment** on **Bob's account** at his bank

Interlude: Grants

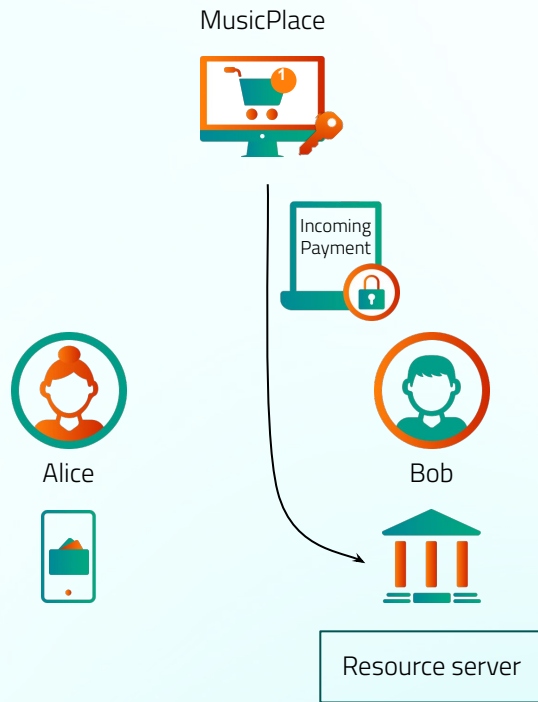
Grant **N**egotiation and **A**uthorization **P**rotocol

- Clients must receive grants before issuing payment instructions.
- Grants give clients the **authorization**, via access tokens, to perform operations.
- All requests require signatures, which protect the integrity of the requests. Signatures are generated according to the HTTP Signatures specification.
- GNAP makes it possible to give account holders specific and **fine-grained control** over the permissions they grant, including control over the amounts of transactions with time-based and velocity-based limits. This enables third-party payment initiation and delegated authorization without compromising the security of the underlying financial accounts and payment instruments.

Open Payments

Third-Party Access to Accounts

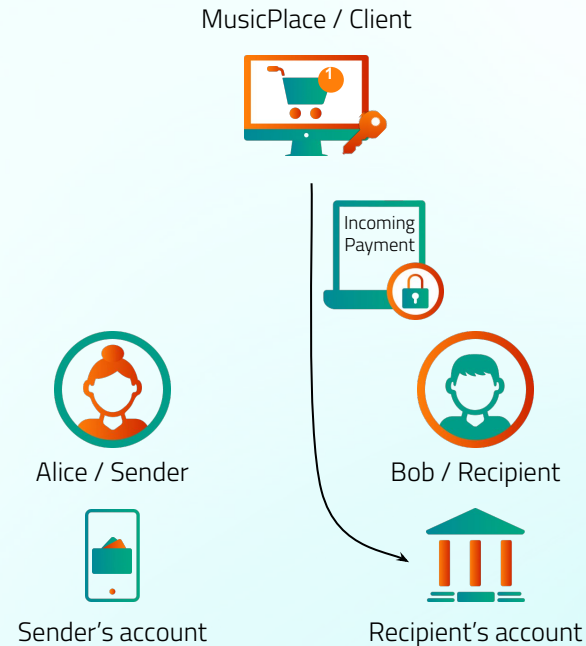
- Grant Request Incoming Payment
- **Create Incoming Payment**
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment



- MusicPlace creates an **Incoming Payment** on **Bob's account** at his bank

Interlude: Incoming Payment

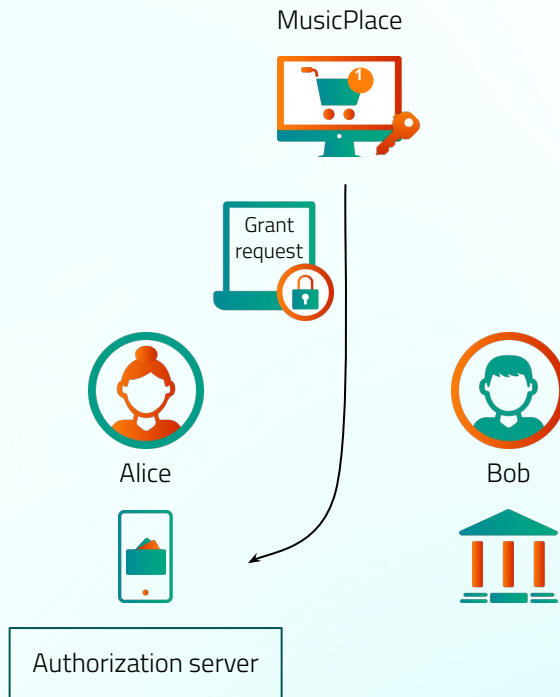
- The process begins by creating an incoming-payment on the recipient's account.
- When created, the recipient's ASE returns unique payment details to the client that can be used to address payments to the account.



Open Payments

Third-Party Access to Accounts

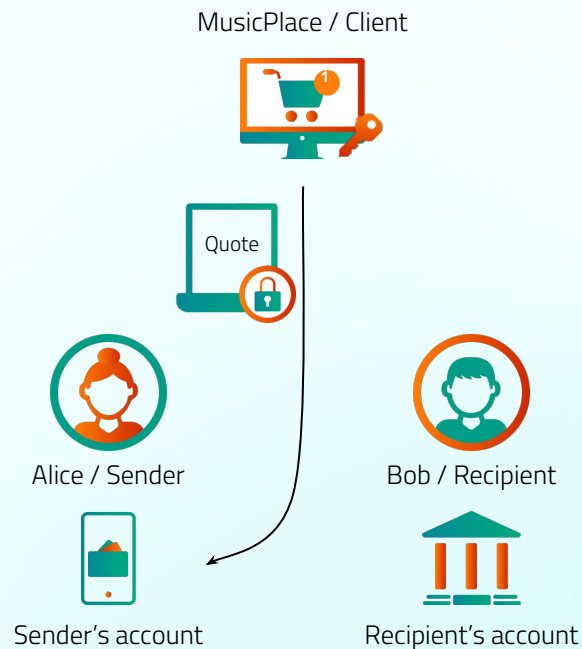
- Grant Request Incoming Payment
- Create Incoming Payment
- **Grant Request Quote**
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment



- MusicPlace requests a **Grant** to create a **Quote** for the payment on **Alice's account** at her wallet provider

Interlude: Quote

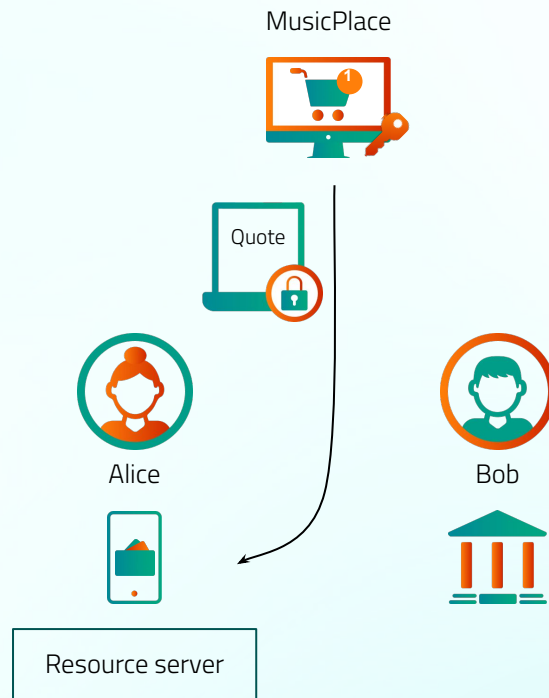
- After an incoming-payment is created on the recipient's account, a quote must be created on the sender's account.
- The purpose of a quote is to indicate **how much the transaction will cost**, including fees.
- The quote is a commitment from the sender's ASE to deliver that amount to the recipient's ASE.
- A quote is only valid for a limited time.



Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- **Create Quote**
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment

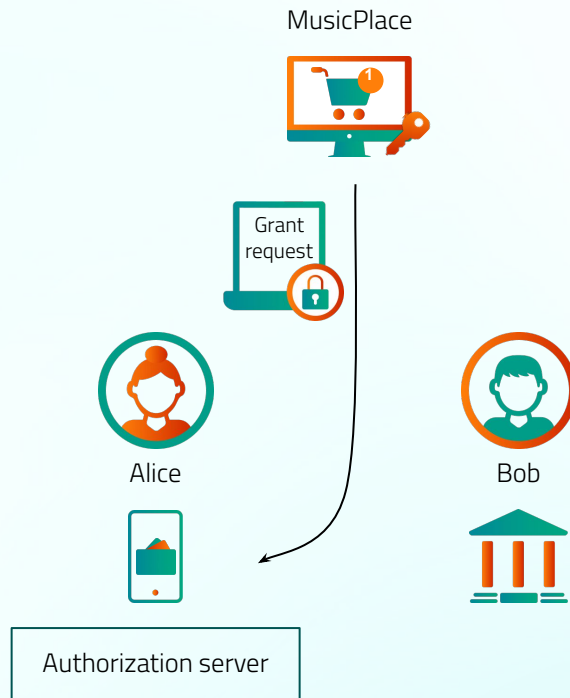


- MusicPlace creates a **Quote** for the payment on **Alice's account** at her wallet provider

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- **Grant Request Outgoing Payment**
- Continuation Request
- Create Outgoing Payment



- MusicPlace requests a **Grant** to create an **Outgoing Payment** on **Alice's account** at her wallet provider

Interlude: Outgoing Payment

- After a quote is created Open Payments requires the sender to **explicitly consent** to the creation of an outgoing-payment.
- Consent is obtained **through an interactive grant**.
- The outgoing-payment serves as an instruction to make a payment from the payer's account.
- The request to create the outgoing-payment includes:
 - ◆ the recipient's wallet address, so the sender's ASE knows where to send the payment
 - ◆ the quote ID, where the payment amounts are defined.

Example Consent Screen

A **combination** of network, exchange, and wallet **fees**



MusicPlace would like to send **USD 1.45** to **Bob** (<https://wallet.com/bob>). This will cost you **EUR 1.36**. Do you agree?

Yes

No



Amount to receive: USD 1.45

Amount in local currency*: EUR 1.33

Estimated** transfer fee***: EUR 0.02

Estimated** wallet fee: EUR 0.01

Total debit amount: EUR 1.36

* exchange rate by ECB, without markup

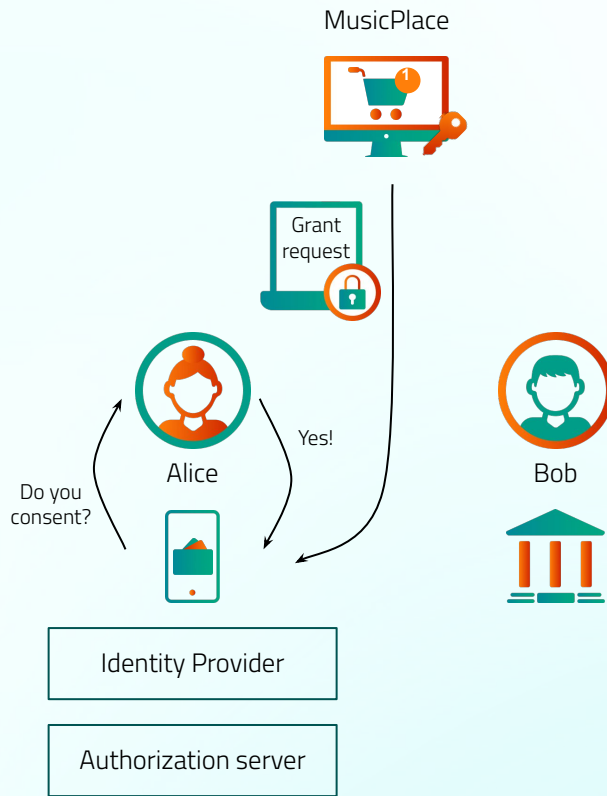
** transfer fees can't be precise due to ILP, will eat into wallet fee

*** transfer fee includes network and exchange fees

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- **Continuation Request**
- Create Outgoing Payment

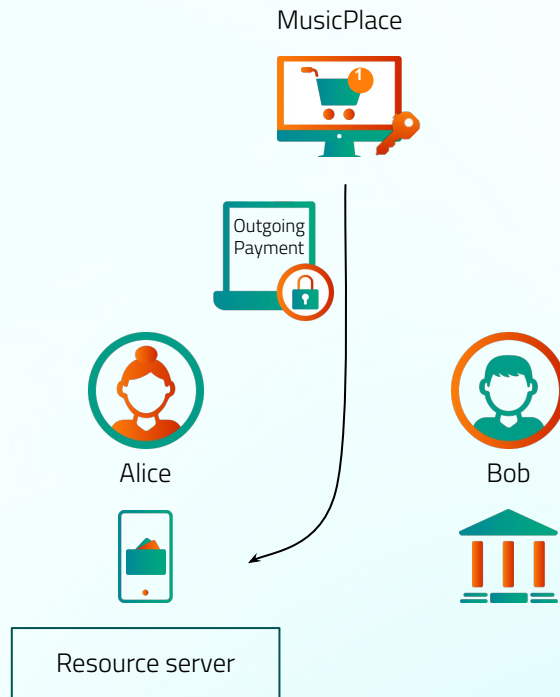


- Alice's wallet provider asks her to **approve** the payment

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- **Create Outgoing Payment**

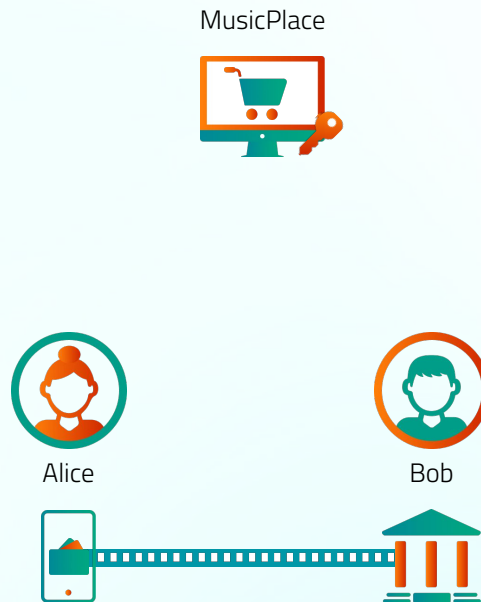


- MusicPlace creates an **Outgoing Payment** on **Alice's account** at her wallet provider

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment

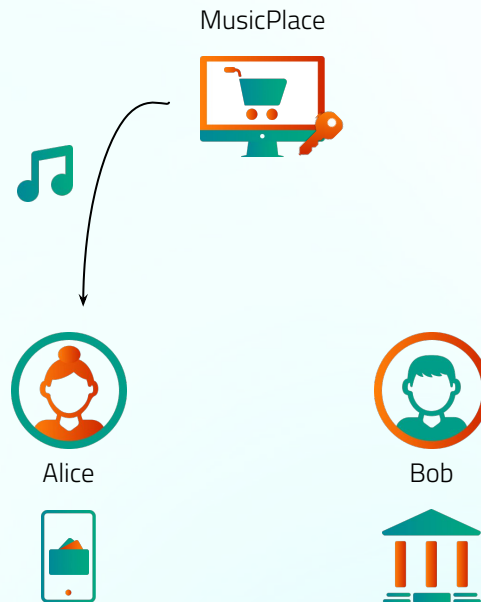


- **Payment** is sent **from Alice's account** at her wallet provider **to Bob's account** at his bank

Open Payments

Third-Party Access to Accounts

- Grant Request Incoming Payment
- Create Incoming Payment
- Grant Request Quote
- Create Quote
- Grant Request Outgoing Payment
- Continuation Request
- Create Outgoing Payment



- Alice can download Bob's music

Deep Dive: Authorization

Grants

- A grant indicates a transfer of authorization from a resource owner (RO) to a piece of software (the client application).
- Grant types match our resource types:
 - ◆ Incoming payments
 - ◆ Quotes
 - ◆ Outgoing payments
- Once the grant is validated, it can be exchanged for access tokens.

Deep Dive: Authorization

Access tokens

- An access token is issued by the authorization server.
- A client will present the access token to a resource server to access protected resources.
- The resource server can verify the validity of the token by making a request directly to to the authorization server.

Deep Dive: Authorization

Why we chose the **G**rant **NA**uthorization **P**rotocol

- OAuth 2.0 dominates today
 - ◆ great for access to information (e.g. "Sign in with Google") using simple scopes.
- But payments = delegating control
 - ◆ need details like amount, limits, frequency. Scopes weren't built for that.
- GNAP is built for actions
 - ◆ lets clients request structured payments (e.g. "\$5 to Alice, monthly") expressed in JSON instead.

Security

- Successful grant requests result in access tokens.
- Access tokens ensure that only authorized operations are executed.
- We're still left with two very important questions:
 1. How do we know we can trust that the client sending the request is who they say they are?
 2. How can we be sure that the request hasn't been tampered with?

Client Keys

- All client requests in Open Payments are signed using a unique key that identifies the client to the authorization and resource servers.
- A client must generate and add its key to a key registry before requesting a grant for the first time. Each client is represented by its own wallet address.
- A key registry is a list of keys associated with client applications.
- Key registries are publicly exposed endpoints. Servers can retrieve the client's key registry by accessing `WALLET_ADDRESS/jwks.json`
- This allows servers to verify that a client is who they says they are.

What does this look like in practice?

Initialize Open Payments client

```
const client = await createAuthenticatedClient({  
  walletAddressUrl: WALLET_ADDRESS,  
  privateKey: PRIVATE_KEY_PATH,  
  keyId: KEY_ID,  
});
```

Developer Keys

 Accounts

 Send

 Request

 Transactions

 Grants

 Settings

Account

Developer Keys 

Revoke Multiple Developer Keys

Account: EUR Account 

- `$ilp.interledger-test.dev/4037e16a`

Generate public & private key

Upload key

- `$ilp.interledger-test.dev/sarah`

Generate public & private key

Upload key

Developer Keys

 Accounts

 Send

 Request

 Transactions

 Grants

 Settings

Account

Developer Keys 

Revoke Multiple Developer Keys

Account: EUR Account 

- `$ilp.interledger-test.dev/4037e16a`

Test Key 

Created August 28, 2025

Key ID

fe96a97c-b941-4b4e-952e-6a9e4af91e15 

Public key

Show

Revoke Test Key

Generate public & private key

Upload key

- `$ilp.interledger-test.dev/sarah`

Generate public & private key

Upload key

Interlude: Asymmetric Keys

In The Open Payments Context

- Each client generates a key pair: a public key and a private key.
- Data encrypted with one key can only be decrypted with the other key.
- Public key is registered with the Account Servicing Entity (ASE) in a key registry.
- Clients sign their request using their private key which is kept secret.
- ASE verifies the signature using the client's public key that is associated with them in the key registry.
- Integrity: Ensures that data cannot be tampered with.
- Authentication: Confirms the identity of the sender.

Interlude: Digital Signatures

Client Side: Creating The Signature

→ Inputs:

- ◆ Request data
- ◆ Client's private key

→ Process:

- ◆ The request data is hashed (converted into a fixed-size string).
- ◆ The hash is encrypted with the client's private key, creating the signature.

→ Output:

- ◆ The signature is a unique encrypted string.

Interlude: Digital Signatures

Server Side: Verifying The Signature

→ Inputs:

- ◆ Received request data
- ◆ Signature from the request
- ◆ Client's public key

→ Process:

- ◆ The server hashes the received request data.
- ◆ The server decrypts the signature using the client's public key to get the original hash.
- ◆ The server compares both hashes. If the hashes match, the signature is valid.

In Summary

- Open Payments enables direct payments from user accounts → no third-party processors, no extra fees, no sensitive data.
- How it works: Both sender & recipient accounts must be OP-enabled
 - ◆ Discovery → query wallet address to find auth & resource servers
 - ◆ Authorization → get scoped access via GNAP
 - ◆ Resource creation → set up incoming payments, quotes, outgoing payments
 - ◆ Settlement → handled by providers outside OP
- The takeaway: A standardized, secure way to build payment apps that are interoperable, privacy-preserving, and user-friendly.

Wallet Setup

- Navigate to **<https://wallet.interledger-test.dev/>**
- Complete the signup and KYC flows
 - ◆ No need to put in real personal information, placeholder values should work
- Create a new (financial) account
- Create a Wallet Address
- Deposit Liquidity